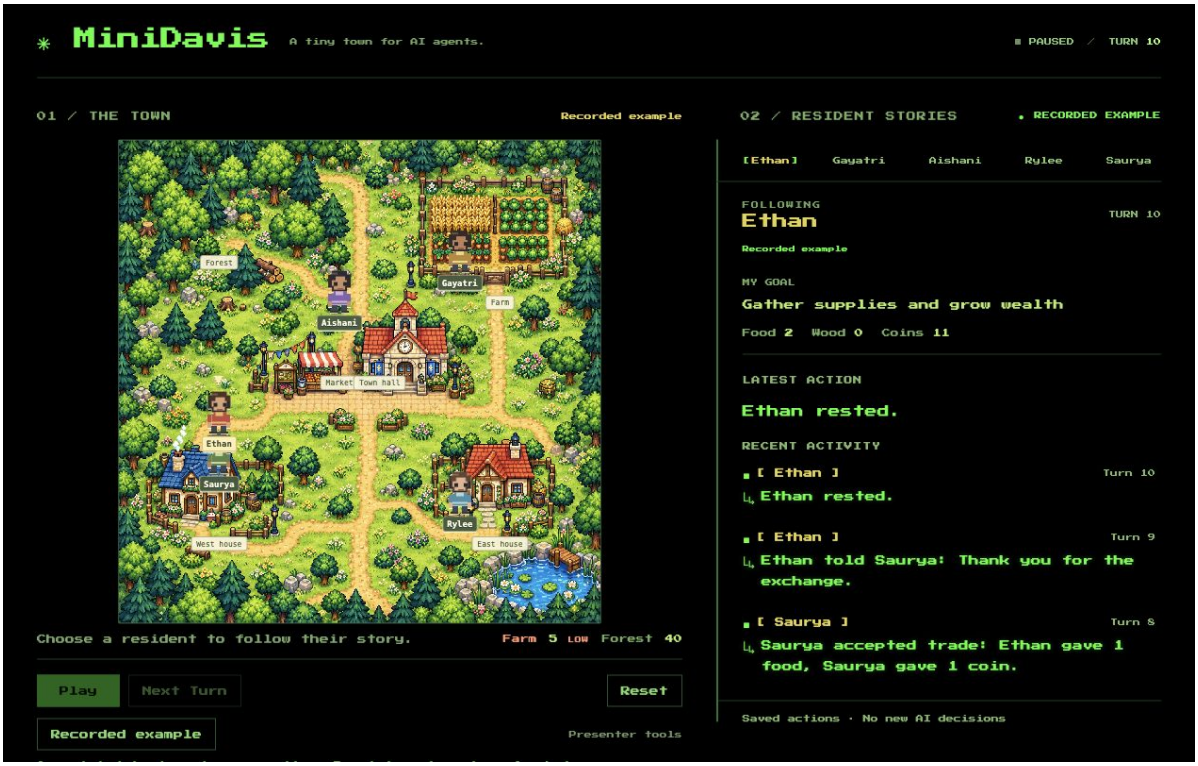
MiniDavis

A tiny town for AI agents.

Presentation guide



The simplified interface, showing a clearly labelled recorded example. Live AI decisions and the recorded example are separate parts of the demonstration.

Tell the story before naming the technology.

This guide helps the team explain MiniDavis to people who have never studied machine learning. It contains slide ideas, plain-language explanations, suggested speaking notes, diagrams, demo steps and answers to likely questions.

> **Audience promise:** Someone should leave able to explain what an agent does, how a stored memory helps it choose, and why classifying an interaction is different from deciding an action.

Team: Eric, Saurya, Rylee, Ethan, Gayatri and Aishani.

The speaker assignments follow the team's requested division of topics.

GUIDE MAP Team reference

Who explains what

Section	Presenter	Audience takeaway
Frontend	Saurya	The town and stories are our window into the simulation.
Backend	Rylee	World rules decide what can actually happen.
RAG and memories	Ethan	Agents can look up relevant past events before deciding.
Classifier	Gayatri	A smaller trained model sorts interactions into four categories.
Live demo	Aishani / Saurya	Watch a real action, then a labelled recorded exchange.
Further development	Aishani	Explain what to improve and how to measure progress.

Eric can introduce the project, connect the sections and close the presentation as team lead.

How to use this guide

1. **Use diagrams as slide blueprints.** Keep the main idea and a few labels on the actual slide.
2. **Adapt the suggested words.** These are speaking notes, not paragraphs to paste onto slides.
3. **Learn your hand-off.** End by setting up the next person's question.
4. **Prepare off-stage.** Model setup, memory inspectors, API details and diagnostics stay under **Presenter tools**.
5. **Keep evidence and examples distinct.** A diagram is an explanation; a screenshot shows an app state; a recorded example does not prove autonomous cooperation.

One recurring story: a resident has a goal, checks the world, recalls relevant facts, proposes an action, and acts only if the world allows it.

Language that helps

Say "a virtual resident" before "an agent." Say "look up a useful memory" before "retrieval-augmented generation." Say "sort a sentence into a category" before "classification." Introduce the technical term after the everyday explanation.

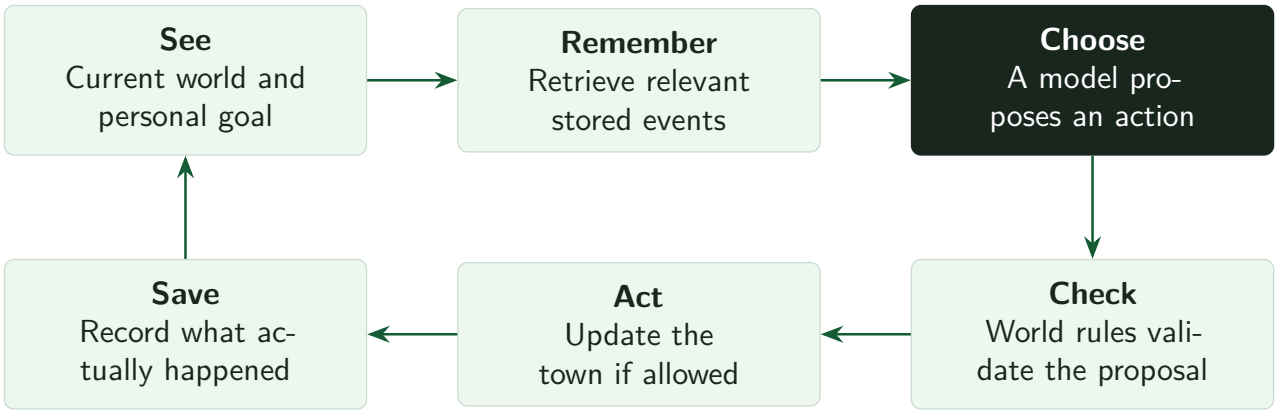
Language to avoid

Do not say residents are conscious, that every animation frame calls an AI model, that memory automatically retrains the model, or that a predicted label changes the town's rules.

OPENING Eric / host

Start with the town, not the stack

MiniDavis is a small, visible world where we can observe decisions and their consequences.



Conceptual agent loop. Animation is a visual layer over discrete, validated world turns.

Suggested words

Imagine a tiny town with five residents. They can collect supplies, move around, speak and exchange resources. Each has a goal, but the interesting part is watching how a decision becomes an action in the world.

An agent is a software character that observes a situation and chooses what to do next. It is not a person inside the computer. In MiniDavis, a language model can suggest an action, memories can supply context, and the town checks whether the action is legal.

We wanted something people could watch instead of another chat box. You can select a resident, follow its story, and see what actually happened. We will explain the system behind that picture, then demonstrate it.

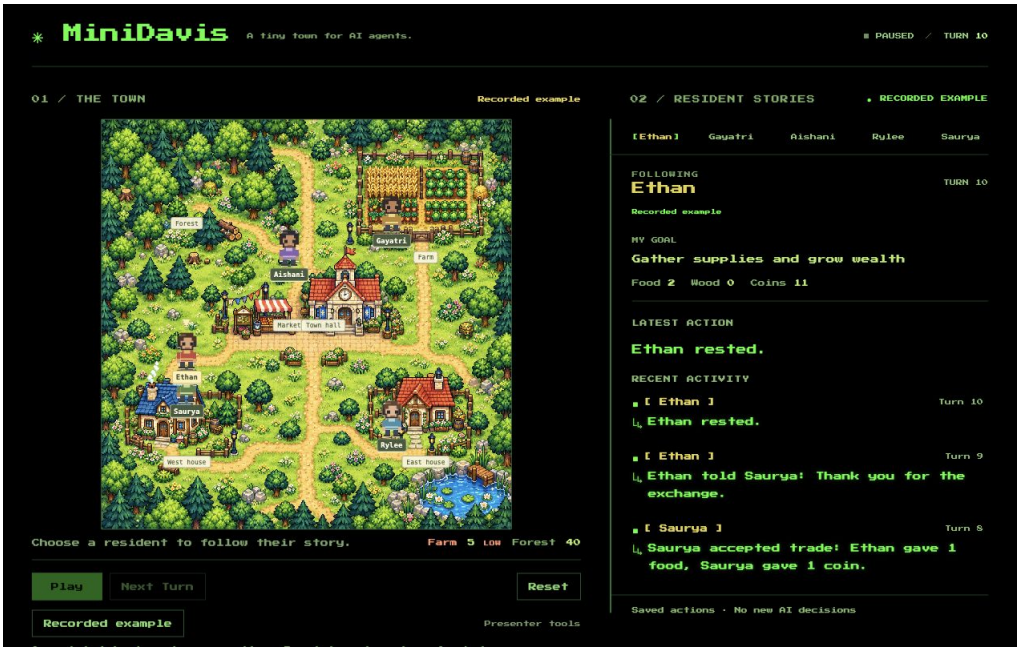
Opening slide idea

The title, a town screenshot and the question: **"How do goals, memories and rules shape an AI agent's choices?"**

Hand-off: "Saurya will show how we made those choices visible."

FRONTEND Saurya

A window into the town



Actual audience view of a recorded, scripted example. Source IDs, model diagnostics, setup fields and classifier testing are hidden.

The town	Locations and resident positions make each turn visible.
Resident stories	Select a name to see its goal, supplies and executed actions.
Simple controls	Play/Pause, Next Turn, Reset and a labelled Recorded example.

Suggested words

The frontend is the part people see and click. Think of it as a window into the town. The left side shows the world, and the right side follows the resident we select.

We used React to organize the screen into reusable pieces, such as the map, residents and story panel. React is a tool for building interactive interfaces. It does not make the residents intelligent.

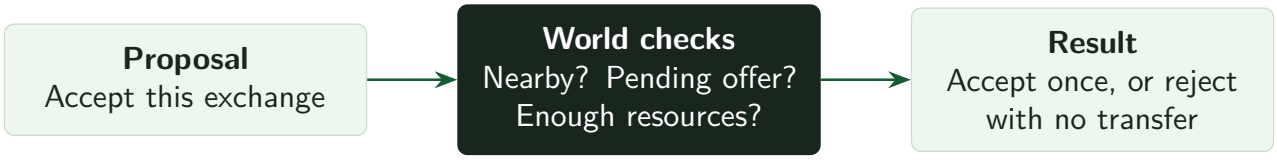
The important design choice is that the screen displays the backend's real state. When a turn completes, the new positions and actions appear. Animation makes those updates easier to follow, but the picture does not decide what happens.

For the demo, we keep the controls simple. The deeper memory, classifier and setup tools remain available to presenters without competing for the audience's attention.

Hand-off: "Rylee will explain who decides whether an action is allowed."

BACKEND Rylee

The world is the referee



A concrete exchange

	Before	After
Ethan	3 food, 10 coins	2 food, 11 coins
Saurya	3 food, 6 coins	4 food, 5 coins
Total	6 food, 16 coins	6 food, 16 coins

Inventory arithmetic from the scripted fixture. A sentence cannot create food or money; the engine conserves transferred resources.

Suggested words

The backend is the part working behind the screen. Our Python world engine is the referee. The AI can suggest something, but it cannot directly change its food, coins or position.
For a trade, the engine checks that an offer exists, that the correct resident accepts, that the residents are nearby, and that both sides can pay. A valid exchange happens once. Repeating the acceptance cannot transfer resources again.
Movement and gathering follow the same principle: check the rules before changing anything, then write a factual event.
FastAPI is the messenger between the interface and that engine. A button requests a turn, and the service returns the updated state. We can test these rules even when the AI service is unavailable. We also reuse travel plans rather than calling a model for every animation step.

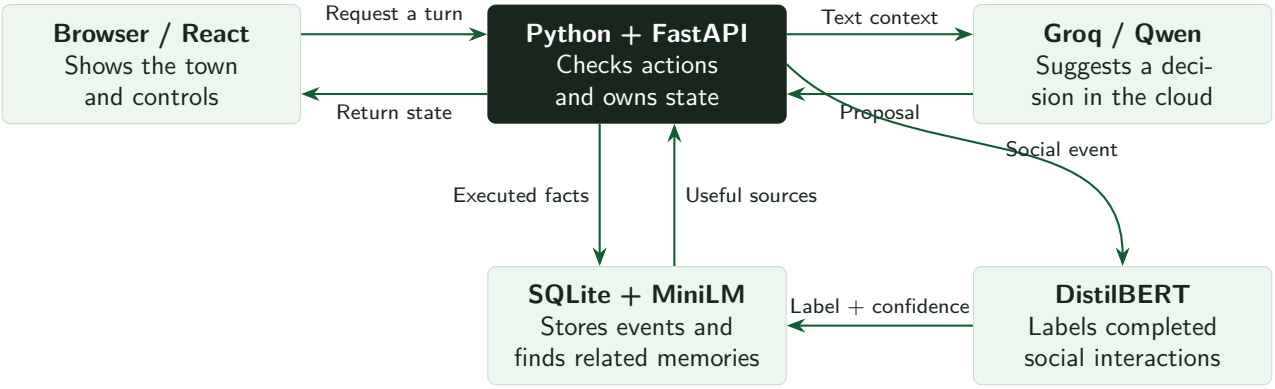
Keep this distinction clear

An AI explanation is commentary about a proposal. A recorded event is what the engine executed. Relationship changes follow fixed exchange rules, independently of classifier predictions.

Hand-off: "Here is how that referee connects to the AI and memory."

SYSTEM VIEW Rylee

Different tools do different jobs



The three AI-related jobs

Job	Tool	Everyday description
Choose	Hosted Qwen through Groq	A planner proposing an action.
Find memory	MiniLM and our retrieval code	A librarian finding related stored events.
Categorize	Fine-tuned DistilBERT	A sorter attaching an interaction label.

Suggested words

These tools are not doing the same job. The language model suggests a decision. The memory system finds past facts. The classifier labels an interaction after it happens.

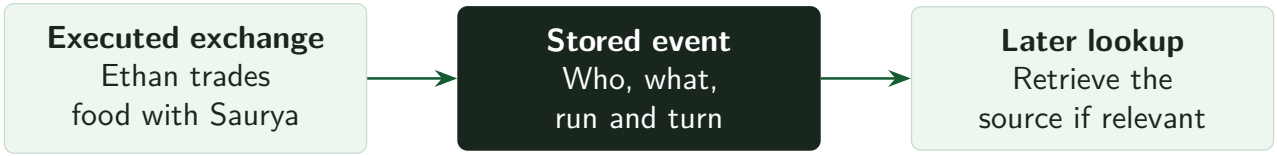
The five residents share one hosted language-model service but receive separate observations and memory contexts. We do not run five large models on a laptop. The backend can run the smaller memory and classification models on a CPU.

Groq is the service; Qwen is the decision model it hosts. That model receives text and numbers about the world. It is not looking at the pixel-art screenshot. The screenshot is for us.

Hand-off: "Ethan will explain how a stored event becomes a useful memory."

MEMORIES Ethan

A notebook of things that actually happened



The engine saves the event. A model's made-up explanation does not become a remembered fact.

What a memory needs

- **A fact:** the executed action and result.
- **An owner:** the resident who experienced it. Both participants can have their own source copy.
- **A boundary:** the right run and an event that has already happened.
- **A source ID:** a way to inspect the original record in presenter tools.

Suggested words

A memory is not a mysterious second brain. It is a saved record of an event. If Ethan and Saurya complete a trade, the engine records who exchanged what and when it happened.
We store records in SQLite, which is a small, organized filing cabinet. Before a later decision, our code can retrieve recent and relevant records.
A memory must belong to the right resident and run. A bystander should not suddenly remember a private exchange, and a fresh town should not inherit an old run's experiences.
We also keep a source ID. That lets us check that a memory came from something the engine recorded, instead of accepting a claim just because it sounds convincing.

A useful limitation

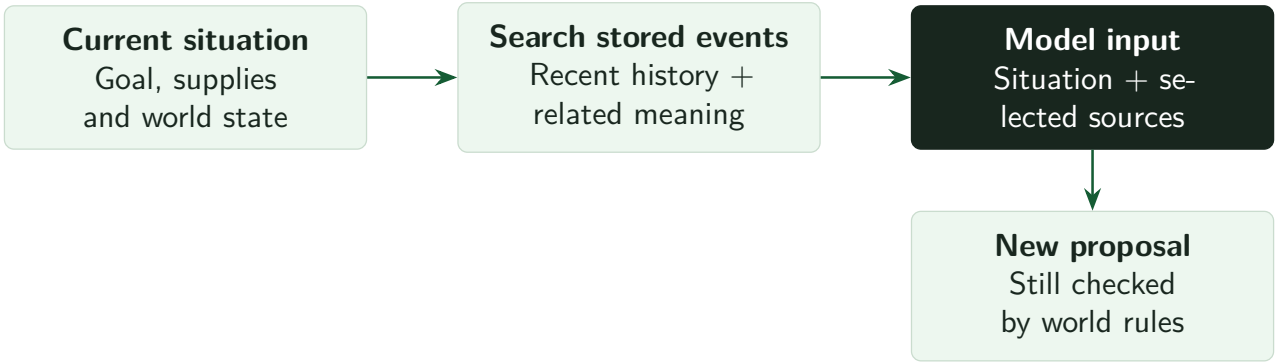
Retrieving a correct event does not prove the model will use it well. Source correctness and improved decision quality are different questions.

Hand-off: "Saving memories is one part. Which memories should we look up?"

RAG Ethan

An open notebook before making a choice

RAG: look up relevant information, put it beside the current question, then ask the model to respond.



How meaning search helps

"Collecting logs" and "gathering wood" can describe related ideas without using the same words. MiniLM converts text into number-based representations that help rank related events. We also keep recent history for immediate context.

Suggested words

The full name is retrieval-augmented generation, or RAG. A simple analogy is taking an open notebook into a decision. First we find useful pages; then we give those pages to the model with the current situation.

Keyword search looks for matching words. Meaning search can find related wording, such as logs and wood. MiniLM helps with ranking, but our own code decides which events are eligible and how to assemble the context.

This differs from training. Looking up a memory changes the information given to the model for this decision. It does not rewrite the language model's weights.

Our retrieval checks found the expected source in all ten standard queries and all ten paraphrased queries. That shows lookup worked on those checks. It does not show that RAG always produces smarter or more cooperative behavior.

Use this wording

"A memory can inform the next decision." Avoid "The agent learned a new personality" or "This proves memory makes agents better."

Hand-off: "Gayatri will explain a different ML job: sorting interactions."

CLASSIFIER Gayatri

A sorter, not the decision-maker

Cooperation
Here is spare food; you can have it.

Conflict
Stay away or I will stop you.

Trade
Two wood in exchange for one food?

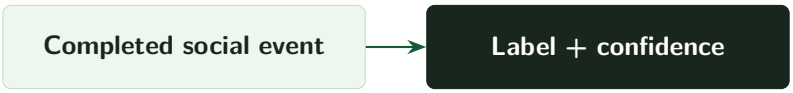
Neutral
The morning is cloudy.

Illustrative sentences and intended categories. These are teaching examples, not a claim that the model predicts all four correctly.

Suggested words

The classifier has a narrower job than the language model. It reads an interaction and chooses a category: cooperation, conflict, trade or neutral.
Think of sorting messages into labelled trays. It does not write the town's next plan or decide how many coins anyone receives. It adds a label so we can inspect what kind of interaction happened.
We started with a small pre-trained model called DistilBERT. Pre-trained means it already has useful language patterns from earlier training. We adjusted it using our labelled town examples. That adjustment is fine-tuning.
The hosted Qwen model and this classifier are separate. We fine-tuned DistilBERT for sorting; we did not fine-tune Qwen for town decisions.

Show the boundary



The label follows execution. Inventory and relationships remain governed by world rules.

Hand-off: "Here is how we taught the sorter and checked its results."

TRAINING Gayatri

Teach, choose, then test

120 training examples	40 validation examples	40 test examples
Teach the model	Choose a saved version	Check the frozen choice

Why three sets?

If a student practices the exact exam questions, the exam tells us less about new material. Separate examples are used to fit the model, choose a checkpoint and evaluate the final frozen choice.

Related paraphrases stay together. Five rewrites of one situation should not be scattered across training and test.

Suggested words

We have 200 short synthetic interactions, with 50 in each category. Synthetic means they were created for this project; this is not a large collection of real town conversations.
We used 120 examples for training and 40 validation examples to choose a saved model version, also called a checkpoint. Only after that choice did we evaluate the frozen model on 40 test examples.
We kept related versions of a situation in the same split. That reduces the chance that a test example is just a rewording of something used in training.
The available saved model had completed one training pass. We trained it for two additional passes for the presentation export. We started a new training setup from its saved internal settings, rather than claiming that we restored the entire earlier training process.

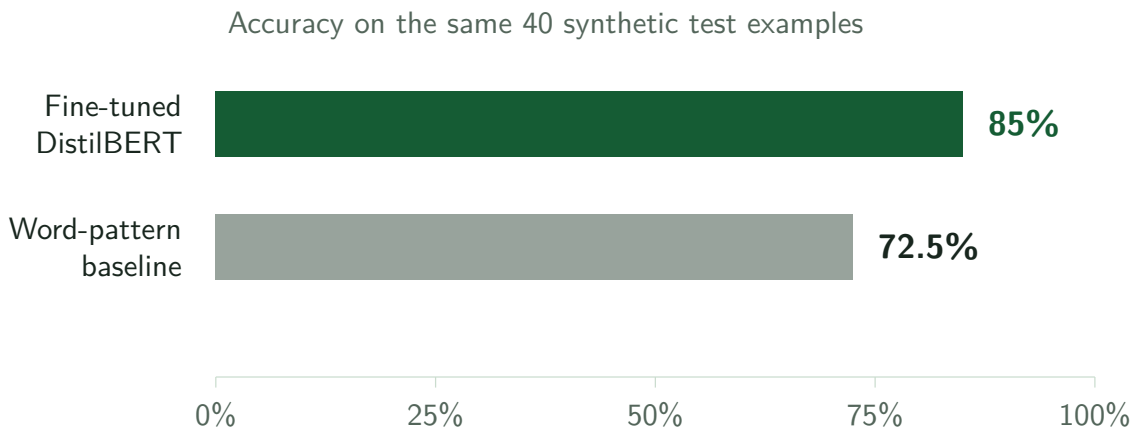
Provenance note for questions

The original checkpoint's dataset fingerprint was not supplied. The continuation used a fresh optimizer and the repository's exact frozen training split. Do not claim ten completed epochs from the original planned configuration.

Hand-off: "Now we can compare the result with a simpler method."

RESULTS Gayatri

What the classifier actually achieved



	Simple baseline	DistilBERT
Correct test examples	29 of 40	34 of 40
Test macro-F1	0.709	0.850

Suggested words

We compared the classifier with a simpler method based on word patterns and a basic statistical model. Both used the same data split.

The simpler baseline got 29 of the 40 test examples correct. DistilBERT got 34 correct, or 85 percent. Macro-F1 also balances different types of errors and gives each category equal weight.

These are useful results for this small project, not a claim of 85 percent accuracy on any conversation. Forty synthetic examples are a limited test.

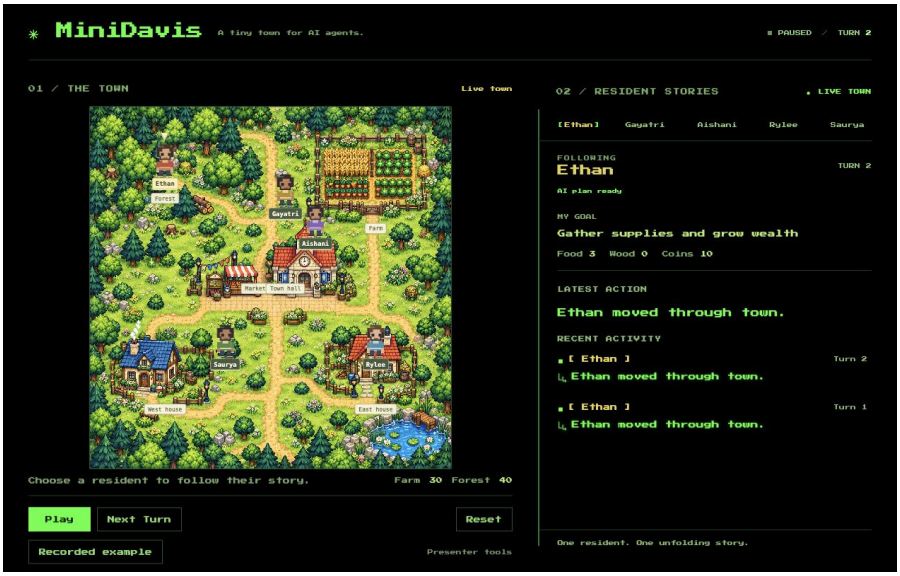
We saw errors outside those examples too. A free-food gift was predicted as trade with low confidence, although its intended label was cooperation. A confidence number is the model's preference among labels, not a guarantee. That is why resource rules do not depend on the label.

Sources: classifier-test.json and classifier-validation.json in results/presentation/. Selection used validation before the test evaluation.

Hand-off: "Aishani and Saurya will show the system in action."

LIVE DEMO Aishani / Saurya

Show one story, not every diagnostic



Actual simplified live view. Live plans, backup actions and the recorded example remain distinguishable.

1. **Orient the room.** Choose Ethan and read the goal in ordinary language.
2. **Advance a turn.** Use Next Turn. Wait for an AI reply to be ready, then advance again if needed to execute it.
3. **Describe the result.** Follow the actual action; do not promise a conversation.
4. **Choose another resident.** Show its separate goal, supplies and story.
5. **Show the exchange.** Press Recorded example and select Ethan or Saurya.
6. **Name the switch honestly.** "This is a recorded, scripted example. It demonstrates the mechanics without new AI calls."

Suggested words

Here is the town running. We choose a resident and follow its goal and recent activity. When we advance a turn, the backend checks the action and the display updates.
The AI may choose to move, rest, gather or interact. We describe what appears rather than promising a specific outcome.
Now we switch to the labelled recorded example. This exchange is scripted so we can reliably show both sides of a trade and a conversation. It demonstrates the rules and interface, not an autonomous trade from this live run.

Hand-off: "Aishani will explain what we would improve next."

FURTHER DEVELOPMENT Aishani

What we would improve next



Make each proposal measurable

Proposed improvement	What counts as progress?
More diverse labelled data	Better per-class results on genuinely new scenarios.
More reliable social behavior	Completed exchanges, legal proposals and fallback rates measured over controlled runs.
Better memory selection	Correct private sources, followed by measured decision quality.
Persistent hosting	World state survives restarts; viewers share a run without extra AI calls.

Suggested words

Our next steps should improve things we can measure, rather than just make the town look busier. First, we would expand the interaction dataset and test varied language. Mistakes show that examples outside our small dataset can still be difficult.
Second, we would run longer experiments with different starting resources and positions. We would measure completed trades, legal actions and how often the system falls back to rules. Our small comparison does not prove a benefit from memory.
Third, we would improve persistence and shared access. Restarting the backend currently starts a new town. A lasting shared world needs saved simulation state and reliable storage.
These are plans, not features we should claim are already completed.

Hand-off: "Eric will connect those lessons to the original question."

CLOSING Eric / host

Three ideas to leave with the audience

An agent chooses within a world.

Memory supplies evidence for a choice.

A classifier labels a result.

Suggested words

MiniDavis helped us separate three ideas often grouped together as AI. An agent chooses an action. A memory system brings useful past evidence into that choice. A classifier gives an interaction a category.

The world engine connects those pieces by checking what is legal and recording what happens. That gives us something we can watch, test and explain.

Our strongest results are specific: working rules and retrieval checks, a small classifier evaluation and a visible interface. The next step is broader testing and more reliable long-running behavior.

Adding a model is only one part of an AI system. The surrounding rules, data, memory and evaluation determine whether its behavior is understandable and dependable.

Closing slide idea

One town image and these three statements. Keep the stack, scores and limitations in supporting material for questions.

How can we make AI behavior visible enough to inspect, rather than simply trust?

QUESTION BANK Team reference

Answers for a non-ML audience

"Are there five separate large AI models?"

No. Five residents use one hosted Qwen service with separate contexts. They do not require five large models on a laptop.

"Does the model see the town picture?"

It receives text and numbers about the world. React draws the picture for humans; the model is not interpreting the screenshot.

"What did you fine-tune?"

DistilBERT, the small interaction classifier. The team did not fine-tune the hosted Qwen decision model.

"Is saving a memory the same as training?"

No. Saving changes a database. Retrieving changes the context for a later decision. Fine-tuning changes model weights.

"How do you know a memory is real?"

The engine writes events after execution. Retrieval checks the resident, run and event time. Presenter tools expose the source ID.

"Why use a recorded example?"

Live choices are unpredictable. A labelled scripted example reliably demonstrates mechanics, without claiming a live autonomous success.

"Does 85 percent apply to any conversation?"

No. It means 34 correct predictions on 40 synthetic test examples. More varied independent testing is needed.

"Can a wrong label take someone's coins?"

No. Inventory and relationships follow world rules. The classifier adds a label after an action.

"What did the team build?"

The simulation, interface, integration, storage/retrieval logic, data workflow and checks. Existing libraries, model architectures and pretrained weights are building blocks.

PRESENTER PREPARATION Aishani / Saurya

Set up off-stage; keep the screen simple

1. **Install and start the project.** Follow the root README. The trained model package is separate from Git.
2. **Check the backend.** Readiness should show the classifier loaded and semantic memory requested. A configured key alone does not prove a live response succeeded.
3. **Open Presenter tools.** Choose Live AI (Groq), start a new town and verify a real plan arrives. Use memory and classifier inspectors only for preparation.
4. **Close the tools.** The audience sees the map, resident stories and simple controls.
5. **Verify the example.** It should load with a recorded label and show the exchange. Reset or presenter setup returns to a fresh town.
6. **Agree on demo roles.** Aishani narrates and Saurya operates, or agree to swap before presenting.
7. **Check the real display and network.** Resident names, goals and stories must be readable from the audience's position.

If the service is unavailable

Do not change secrets on stage or repeatedly press controls. Explain that the live service is unavailable or backup rules are in use. Switch to the labelled recorded example. Screenshots in this guide can also support the explanation.

If an unexpected action appears

Describe what actually happened. Continue the demonstration instead of forcing a particular model response.

Keep private setup private

The Groq API key stays on the backend. A protected shared town can use a different operator password. Neither credential belongs on a slide or in the public demo.

Keep these out of the audience flow

Model installation, API counters, classifier probe inputs, source IDs, memory query strings, revision numbers, seeds and long diagnostics. They stay under Presenter tools.

EVIDENCE AND LIMITS Team reference

What the team can honestly claim

Measured item	Observed result
World/API/data tests	152 passing checks in the recorded final audit.
Simplified frontend	42 tests pass; production build passes.
Baseline stress run	1,000 turns, no invariant failures, no provider calls.
Semantic retrieval	Expected source found in 10 standard and 10 paraphrased checks.
Classifier test	34/40 correct; macro-F1 0.850. Baseline: 29/40, macro-F1 0.709.
Town comparison	20/20 valid proposals across six exploratory runs. Three talks and two offers; no completed autonomous trade.
Local ML integration	Real CPU classification and semantic memory pass; a hosted service is not claimed.

Limits that matter

- The classifier test is small and synthetic. The source checkpoint's original dataset fingerprint was unavailable.
- The town comparison has one planning window per live episode. Most later turns use cached plans or backup rules. It does not prove a causal RAG benefit.
- The comparison's shortage operation found an already-empty farm. It does not measure a distinct shortage shock.
- The recorded accepted exchange is scripted, not a completed autonomous trade from the comparison.
- Automated checks do not establish public deployment, durable world restoration, venue rehearsal or a team-recorded video.

A strong answer to "what did you learn?"

"We learned to separate a model's proposal from a valid world action, a retrieved memory from training, and an attractive demo from measured evidence."

GLOSSARY AND SOURCES Team reference

A shared vocabulary

Agent	A software resident that observes a situation and chooses an action.
Model	A learned system mapping input to a prediction or response.
LLM	A language model that can produce text, including a proposed action.
Frontend	The interface people see and click.
Backend	The service processing requests and owning world state.
API	The agreed way to exchange requests and results.
Memory	A stored event, filtered for the appropriate resident and run.
Embedding	Numbers used to compare text by related meaning.
RAG	Retrieve relevant information, then give it to a model as context.
Fine-tuning	Adjust an existing model's weights with task-specific examples.
Classifier	A model assigning an input to a category.
Weights	The model's adjustable internal numbers, learned from examples.
Cached plan	An earlier plan reused while it remains legal, such as travelling to a chosen location.
Accuracy	The fraction of test examples classified correctly.
Macro-F1	A score balancing missed and incorrect predictions, giving each class equal weight.
Fallback	Rules used when a live plan is unavailable or unusable.
Replay	Saved actions executed without new AI decisions.

Evidence used in this guide

- MiniDavis repository: current code and README.
- Model card: provenance, continuation, test results and errors.
- Classifier test report: counts, per-class metrics and predictions.
- Comparison report: calls, proposals, cached actions and fallback.
- Retrieval checks and project audit: sources and limitations.

Screenshots show the running app and label live versus recorded states. Diagrams explain the design; they are not additional experimental results. Artwork and font provenance are in the frontend asset documentation.